

人工智能基础与应用 · 大模型实训

M1 · 从 n-gram 到 Transformer

从零实现大模型：M0-M9 知识主干（16 周 / 11 课题 / 零预算）

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

Markdown 母版：人工智能基础与应用/讲义/ 下的 .md 文件 · 配套上游：CS336 A1-A5 · nanochat · minimind

讲义 M1 · 从 n-gram 到 Transformer

对应课题：课题 02 (W2) 前半 | 模块：M1 | 预计 1 个时段 对标材料：李沐 d2l ch8–ch10 (RNN → 注意力) · CS336 lecture 01 (概览) · 《Attention Is All You Need》原文 (走论文精读) **为什么讲这段**：不理解 attention 之前的世界，就不知道它在解决什么问题——而“知道它在解决什么问题”，是你将来能判断“某个新架构值不值得看”的唯一依据。

〇、本讲要回答的 5 个问题

1. n-gram 语言模型为什么必然失败？它的失败方式是什么？
2. 困惑度 (perplexity) 为什么能当指标？它和 M0 的交叉熵什么关系？
3. RNN/LSTM 解决了什么，又为什么被淘汰？
4. seq2seq + attention (2014) 与 Transformer (2017) 的关键差别在哪？
5. 为什么说“并行性”是 Transformer 取胜的主因，而不是“注意力更聪明”？

一、n-gram：用计数做语言模型

链式法则： $P(w_1 \dots w_T) = \prod_t P(w_t | w_{<t})$ 。**n-gram 的近似**：只依赖前 $n - 1$ 个词， $P(w_t | w_{t-n+1} \dots w_{t-1})$ ，用语料计数估计：

$$P(w_t | w_{t-1}) = \frac{c(w_{t-1}w_t)}{c(w_{t-1})}$$

两个死穴（这就是“为什么历史必然翻篇”）：

死穴	具体表现
稀疏	没见过的组合概率为 0 → 平滑 (add-k / 回退 / Kneser-Ney) 只是打补丁
上下文太短	$n = 5$ 时也只能看 4 个词；而“这句话的主语是谁”可能要跨 50 个词

评价指标：困惑度 $PPL = \exp\left(-\frac{1}{T} \sum_t \log P(w_t | w_{<t})\right)$ —— 正是 M0 里交叉熵取指数。**PPL 越低，模型对下一个词越不“惊讶”**。这个指标一路沿用到今天 (bpb 是它的分词器无关版本)。

二、词向量：从 one-hot 到分布式表示

one-hot 的问题：任意两词的内积都是 0 —— “猫”和“狗”的距离与“猫”和“民主”一样远。语义无从谈起。

word2vec (skip-gram + 负采样)：用一个“给定中心词预测上下文词”的浅层网络，训练出的**中间层权重**就是词向量。结果是著名的**向量算术**： $\overrightarrow{king} - \overrightarrow{man} + \overrightarrow{woman} \approx \overrightarrow{queen}$ 。

关键转折：人们发现“预测下一个词”这个任务逼着模型学出**语义结构**。这正是后来 GPT 路线的思想雏形：**语言建模本身就是最好的自监督任务**。

三、RNN / LSTM：把“历史”压进一个向量

RNN： $h_t = \tanh(W_h h_{t-1} + W_x x_t)$ ，用 h_{t-1} 携带历史。**LSTM**：加门控（遗忘门、输入门、输出门）让信息可以“长期保留”。

为什么被淘汰（两个根因，都是工程性的）：

1. **梯度消失/爆炸**: BPTT 要对时间展开求导, 梯度是 $\prod_t \frac{\partial h_t}{\partial h_{t-1}}$ 的连乘。连乘里每个 Jacobian 的谱半径 $<1 \rightarrow$ 指数衰减; $>1 \rightarrow$ 爆炸。这是结构性问题, 不是调参能解决的。
2. **无法并行**: h_t 依赖 h_{t-1} , 必须按时间顺序串行计算。GPU 有上万核心却只能一次算一步 —— 训练吞吐被“时间”卡死。

这是本讲最重要的一句话: 深度学习的历史不是“模型更聪明”的历史, 而是“让更多算力能有效地作用于更多数据”的历史。Transformer 赢在能并行。

四、seq2seq 与注意力 (2014): 注意力的真正起源

seq2seq 结构: 一个 RNN 把源句编码成一个固定长度向量, 另一个 RNN 从它解码出目标句。

瓶颈: 整句信息被压进一个向量 —— 句子越长, 丢得越多。

Bahdanau 注意力 (2014) 的解法: 解码每一步时, 回头扫描编码器的所有隐状态, 算出“当前该关注哪几个位置”, 加权求和作为当前步的上下文。→ 这就是 **QKV 的雏形** (解码器状态=Query, 编码器隐状态=Key/Value)。

关键历史结论: 注意力最初是给 RNN 打补丁的, 不是 Transformer 的胜利品。Transformer 的贡献是: “既然注意力这么好用, 那还要 RNN 干什么?” —— 把 RNN 整个删掉, 只用注意力 + 前馈层, 于是整段序列可以并行计算。

路径长度对比 (理解“为什么更快”的关键):

架构	任意两位置的依赖路径	并行性
RNN	$O(L)$ (要穿过 L 步)	✗ 串行
Transformer	$O(1)$ (一步直达)	✓ 全并行

代价是注意力本身 $O(L^2)$ 的计算量 —— 这是“用算力换并行”的经典交易, 也是后来 flash attention、稀疏注意力、线性注意力等一整条研究线的起点。

五、常见误解

误解	更正
“注意力是 Transformer 发明的”	2014 年 Bahdanau 就用于 seq2seq; 2017 只是去掉 RNN 并改成 self-attention
“self-attention 与 attention 是两回事”	同一机制, 只是 Q/K/V 都来自同一序列
“RNN 不行是因为效果差”	在长依赖上确实差, 但致命的并行性才是被淘汰主因
“PPL 越低模型越强”	只在同分词器、同数据下可比 (见 M2)

六、能立刻跑的实验

```
# 用 n-gram 的视角感受“上下文长度”的代价: 统计 bigram 的稀疏程度
环境/.venv/bin/python - <<'PY'
import collections, re
text = open("上游/assignment1-basics/tests/fixtures/tinystories_sample.txt",
encoding="utf-8").read().lower()
words = re.findall(r"[a-z]++", text)
uni = collections.Counter(words)
```

```

bi = collections.Counter(zip(words, words[1:]))
print(f"词数={len(words)} 不同词={len(uni)} 不同的相邻词对={len(bi)}")
print(f"→ 光是一个小语料，二元组就已经接近词数的 {len(bi)/len(uni):.1f} 倍；三/四元组的稀疏会爆掉")
print(f"→ 换成长度更大的语料，这个比值会继续上升（稀疏问题的定量感受）")
PY

```

七、代码级提问（3–5 道）

1. 【推导】为什么 BPTT 的梯度是 Jacobian 连乘？写出两步的式子，并说明“谱半径 < 1 就衰减”的含义。
2. 【对比】用一句话说清：Transformer 相比 RNN，用掉了什么、换来了什么？
3. 【判断】如果有一天硬件变成“串行极快、并行极慢”，Transformer 还会是主流吗？说理由。
4. 【联系】d2l 里 attention 的示意图和 M3 讲的 QK^T 是一回事吗？把两者对应起来。
5. 【指标】你的 n-gram 模型 PPL=120，另一个模型 PPL=80，但用的是不同分词器。能说后者更强吗？

八、记忆锚

- 隐喻：RNN = “传话游戏”——一句话传到第 50 个人已经面目全非；注意力 = “所有人同时看着原始白板”——谁离得远都不影响看清原文。
- 口诀：“n-gram 死于稀疏、RNN 死于串行、注意力先治 RNN、Transformer 删掉串行”
- 符号位置法：把 $O(L)$ 与 $O(1)$ 并排写在卡片上——左边是“要走 L 步”，右边是“一步直达”；下面是代价 $O(L^2)$ ，提醒你注意力不是免费的。